

Handover — n8n Training Workflows

October 2026

Overview

This handover covers six n8n workflows built to introduce students to the core n8n building blocks, one idea at a time. Workflows 1 to 5 are small (3 to 7 nodes each); Workflow 6 is slightly bigger and calls three sub-workflows. Every workflow starts from a webhook, so each one can be triggered and tested from Postman.

#	Workflow	What it shows	Nodes used
1	Webhook Intake & Save	Receive form data, clean it, store it	Webhook, Edit Fields, Supabase, Respond to Webhook
2	Router by Category	Send data down different paths	Webhook, Switch, Edit Fields, Respond to Webhook
3	Lookup & Conditional Email	Look a record up, then branch	Webhook, Supabase, IF, Gmail
4	Send Confirmation Email	Build and send an email, then log it	Webhook, Edit Fields, Gmail, Supabase, Respond to Webhook
5	External API Call	Call an outside service and reshape the answer	Webhook, HTTP Request, Code, Respond to Webhook
6	Pension Enquiry (Main + 3 subs)	Break one process into reusable sub-workflows	Webhook, Edit Fields, Supabase, Execute Workflow, Gmail, Code, Respond to Webhook

Together they cover: webhook, data transformation, Edit Fields, router (Switch), conditional (IF), Gmail, Supabase, API integration (HTTP Request) and sub-workflows.

All workflows use only two new database tables, `training_submissions` and `training_events`. No workflow here reads from, writes to or alters any existing production table.

Before you start

The workflows were built for n8n version 2.13.4 and need one database setup step and two credentials.

1. Database setup (Supabase)

Run the file `00_supabase_setup.sql` once in the Supabase SQL Editor. It only creates two new tables and never alters, deletes or updates anything that already exists. It is safe to run more than once.

Table	Purpose	Columns
training_submissions	Rows saved by the intake workflows	id, created_at, full_name, email, category, message, status (default new)
training_events	A simple activity log	id, created_at, submission_email, event_name, event_detail

The script also adds one sample row (an email address you control) so that Workflow 3 has something to find. Replace that address with a real inbox before teaching, otherwise the "found" email goes nowhere.

2. Credentials in n8n

- **Supabase account:** used by Workflows 1, 3, 4 and 6.
- **Gmail account** (OAuth2): used by Workflows 3, 4 and 6.

If a node shows a red warning after import, open it and pick the right credential from the dropdown. This is a one-time step per node.

3. Test URL or production URL

Every webhook has two addresses. Only the path at the end changes between workflows.

Mode	URL pattern	When it works
Test	https://<your-n8n>/webhook-test/<path>	Only while you click Listen for test event in the editor. Fires once per click, and shows data on every node. Best for teaching.
Production	https://<your-n8n>/webhook/<path>	Only after the workflow is published (active).

4. Sending a test from Postman

1. Set the method to `POST`.
2. Paste the test URL.
3. Open Body, choose raw, and set the type to JSON.
4. Paste the JSON from the workflow's section below and press Send.

If the data arrives empty, the body was probably sent as Text or in the Params tab instead of raw JSON.

5. Reading expressions

Anything inside double curly braces is an expression:

- `$json` is the data of the current item entering this node.
- `$json.body.email` is the `email` field you sent. A webhook places everything you send inside `body`.
- `$('#Node Name').item.json.x` reaches back to an earlier node's output. It is needed when a node in between (Gmail, HTTP Request, Supabase) has replaced the data.
- `a || 'default'` uses `a`, or the default when `a` is empty.

Workflow 1: Webhook Intake & Save

Receives form data, cleans it, saves it as a row in the database and replies with the new row's id. Real-world example: a website contact form saving into a database.

Teaches: Webhook, Edit Fields (data transformation), Supabase insert, Respond to Webhook.

Flow: Webhook, then Edit Fields, then Supabase insert, then Respond to Webhook.

1. **Webhook: Intake.** Method `POST`, path `training-intake`, response mode set to Using Respond to Webhook Node, so the caller waits for the final reply. Receives name, email, category and message.
2. **Edit: Clean Fields** (Edit Fields). Creates four fields and drops everything else (headers, raw request data):
 - `full_name`: the name, or "Unknown" if missing.
 - `email`: trimmed and lowercased, so `" ASHA@Example.com "` becomes `asha@example.com`.
Normalising the email now makes later lookups match.
 - `category`: the category, or "general" if missing.
 - `message`: the message, or empty text.
3. **Supabase: Insert Submission.** Operation Create a Row on table `training_submissions`. Each column is mapped to the matching cleaned field. The database fills in `id`, `created_at` and `status` (`new`) by itself. The output is the new row, including its `id`.
4. **Respond: Success** (Respond to Webhook). Responds with JSON containing `ok`, `saved_id` and the cleaned email.

Test (POST to `/training-intake`):

```
{
  "full_name": "Priya Sharma",
  "email": " PRIYA@Example.com ",
  "category": "sales",
  "message": "Interested in your plans"
}
```

Expected reply: `{ "ok": true, "saved_id": 2, "email": "priya@example.com" }`, and a new row in `training_submissions`. The messy email in the test is deliberate: it shows the Edit Fields node doing its job.

Key lesson: clean data once, early, so every later step can trust it.

Workflow 2: Router by Category

Reads the enquiry's category and sends it down the matching branch. Real-world example: support questions go to the support queue, sales questions to the sales team, everything else to a general inbox.

Teaches: Webhook, Switch (router), several Edit Fields branches that merge back into one reply.

Flow: Webhook, then Switch, which splits into three branches (Support, Sales, General), and all three join at one Respond to Webhook.

1. **Webhook: Enquiry.** Method `POST`, path `training-router`, response mode set to Using Respond to Webhook Node. Receives `{ "category": "..."} .`
2. **Router: By Category** (Switch, Rules mode). Compares `category` against fixed values, ignoring upper and lower case:
 - Output 1: equals `support` .
 - Output 2: equals `sales` .
 - Fallback output (extra): anything else, including no category. This keeps unexpected values from being lost.
3. **Edit: Support Queue, Edit: Sales Queue, Edit: General Queue** (three Edit Fields nodes). Each sets `route` and `queue` (Tier-1 Support, Sales Team, General Inbox). Only one branch runs per request.
4. **Respond: Routed** (Respond to Webhook). All three branches connect into this single node. It replies with `routed_to` and `route` .

Tests (POST to `/training-router`, send each in turn):

```
{ "category": "support" }
```

```
{ "category": "sales" }
```

```
{ "category": "billing" }
```

Expected replies: `Tier-1 Support`, `Sales Team`, and `General Inbox` (the fallback). In the editor the highlighted path changes with each request, which is the visual point of the lesson.

Key lesson: an IF node has two exits (true or false); a Switch can have as many exits as you need, plus a fallback.

Workflow 3: Lookup & Conditional Email

Looks an email address up in the database, then sends one email if it was found and a different one if it was not. Real-world example: "check whether this person exists, then reply accordingly".

Teaches: Supabase read, IF (conditional), Gmail on two branches.

Flow: Webhook, then Supabase lookup, then IF (found?), which sends either Gmail "found" or Gmail "not found".

1. **Webhook: Lookup Request.** Method `POST`, path `training-lookup`. It uses the default response mode, so the caller immediately gets "Workflow was started" while the work continues in the background. Receives `{ "email": "..."} .`
2. **Supabase: Find by Email.** Operation Get Many on `training_submissions`, filter `email` equals the incoming email (trimmed and lowercased), limit 1. **Always Output Data is switched on.** Without it, a search with no result produces no items and the workflow silently stops, so the "not found" email

would never be sent. With it on, the node outputs one empty item and the IF still runs.

3. **IF: Member Found?** Condition: `$json.id` is not empty. A found row has an `id` and goes to the true branch; the empty item has none and goes to the false branch.
4. **Gmail: Found** (true branch). Sends to the address in the database row, subject "We found your record", with the person's name and category in the body.
5. **Gmail: Not Found** (false branch). Sends to the email from the original webhook, because the empty item has no address. It reaches back with `${'Webhook - Lookup Request'}.item.json.body.email`.

Tests (POST to `/training-lookup`):

Found branch. Use the email address of the sample row from the setup script:

```
{ "email": "your-email@example.com" }
```

Not found branch:

```
{ "email": "nobody@example.com" }
```

Expected: Postman shows `Workflow was started` both times. The first request sends the "found" email to the real inbox; the second runs the false branch (that address does not exist, so the email will bounce, but the path is visible in the editor).

Key lesson: the IF node decides, and Always Output Data is what lets "nothing found" become a normal path instead of a dead end.

Workflow 4: Send Confirmation Email

Builds a welcome email from signup data, sends it through Gmail, records that it was sent, and replies. Real-world example: a confirmation email after registration, with an audit trail.

Teaches: Edit Fields to build email content, Gmail send, Supabase logging, and reading data from an earlier node.

Flow: Webhook, then Edit Fields, then Gmail, then Supabase log, then Respond to Webhook.

1. **Webhook: New Signup.** Method `POST`, path `training-confirm`, response mode set to Using Respond to Webhook Node. Receives name and email.
2. **Edit: Build Message** (Edit Fields). Prepares everything the email needs in one place: a clean `email`, a `full_name` (or "there" if missing), a `subject` such as "Welcome, Sam!", and a multi-line `body`.
3. **Gmail: Send Confirmation.** Sends to `{{ $json.email }}` with the prepared subject and body, as plain text. Its output is Gmail's own receipt (message id and thread id), so **the original fields are no longer in `$json`**.
4. **Supabase: Log Event.** Creates a row in `training_events`. Because `$json` now holds the Gmail receipt, it reaches back to the Edit Fields node to read the email address. It writes `event_name` "Confirmation Sent" and an `event_detail` sentence naming the recipient.
5. **Respond: Done** (Respond to Webhook). Replies with `ok` and `sent_to`.

Test (POST to `/training-confirm`). Use an inbox you can check:

```
{
  "full_name": "Sam",
  "email": "your-email@example.com"
}
```

Expected: the reply `{ "ok": true, "sent_to": "your-email@example.com" }`, a welcome email in that inbox, and a new row in `training_events`.

Key lesson: after Gmail, HTTP Request or Supabase, `$json` becomes what that node returned. To use earlier data, call `${'Node Name'}.item.json`. This is the most common stumbling block for beginners.

Tip: n8n appends "sent automatically with n8n" to Gmail messages by default. Turn this off in the Gmail node under Options, Append n8n Attribution.

Workflow 5: External API Call

Takes a first name, asks a public web service to guess the typical age for that name, reshapes the answer and replies. Real-world example: calling any outside system from n8n.

Teaches: HTTP Request (API integration) and the Code node (JavaScript data transformation).

What the API is. Agify (<https://api.agify.io>) is not a platform like Google Sheets or Twilio. There is no account, login, dashboard or API key, and n8n has no built-in node for it. It is just one web address that answers questions automatically. You send a name, and it replies with the statistically typical age of people with that first name. It is a guess from public data, not a fact about any individual. That is exactly why it suits teaching: because no ready-made node exists, the workflow must use the general HTTP Request node, which can connect n8n to any service that has an API.

Flow: Webhook, then HTTP Request, then Code, then Respond to Webhook.

1. **Webhook: Ask.** Method `POST`, path `training-api`, response mode set to Using Respond to Webhook Node. Receives `{ "name": "..." }`.
2. **HTTP: Call Public API** (HTTP Request). Method `GET`, URL `https://api.agify.io/`, with one query parameter `name` set to `{{ $json.body.name || 'maria' }}` (a fallback if no name is sent). Authentication: none. The actual call becomes `https://api.agify.io/?name=james`. It returns something like `{ "count": 12345, "name": "james", "age": 58 }`, where `age` is the predicted age and `count` is how many people the estimate is based on.
3. **Code: Shape Result** (Code node, JavaScript, Run Once for All Items). Turns the raw reply into a tidy result:
 - `name`: copied from the reply.
 - `predicted_age`: the age, or null if the API had no answer.
 - `bracket`: `young` under 30, `mid` under 55, otherwise `senior`; `unknown` when there is no age.
 - `sample_size`: the `count`.
 - Note: a Code node must always return an array of items, each wrapped in `json`.
4. **Respond: Result** (Respond to Webhook). Returns the whole shaped object.

The code in step 3:

```
// Transform the raw API response into a tidy result
const r = $input.first().json;
const age = r.age ?? null;
return [{
  json: {
    name: r.name,
    predicted_age: age,
    bracket: age === null ? 'unknown' : age < 30 ? 'young' : age < 55 ? 'mid' : 'senior',
    sample_size: r.count ?? 0
  }
}];
```

Test (POST to `/training-api`):

```
{ "name": "james" }
```

Expected reply: `{ "name": "james", "predicted_age": 58, "bracket": "senior", "sample_size": 12345 }` (the exact numbers vary). Try other names to show the result changing.

Limit: the free tier allows roughly 100 names per day per IP address, after which it returns `429 Too Many Requests`. That is plenty for a class, but do not call it in a loop.

Key lesson: use Edit Fields for simple renaming and defaults; use a Code node when you need if/else logic or calculations. To integrate a different service, change the URL and parameters: the pattern stays the same.

Workflow 6: Pension Enquiry (Main + 3 sub-workflows)

A slightly bigger workflow that receives a member's pension enquiry, saves it, and then hands the follow-up work to three separate sub-workflows. Real-world example: a pension administration team acknowledging an enquiry, alerting an admin and closing the record.

Teaches: Execute Workflow (sub-workflows), passing data between workflows, and a Supabase insert followed by an update of the same row.

Why sub-workflows. Each job (email the member, alert the admin, update the record) lives in its own small workflow. The main workflow stays short and readable, and each piece can be reused or changed on its own.

Flow: Webhook, then Edit Fields, then Supabase save, then Sub 1 (email member), then Sub 2 (notify admin), then Sub 3 (update status), then Respond to Webhook.

The main workflow: Training 6, Pension Enquiry (Main)

1. **Webhook: Member Enquiry.** Method `POST`, path `training-pension-enquiry`, response mode set to Using Respond to Webhook Node.

2. **Edit: Prepare Enquiry** (Edit Fields). Trims and lowercases the fields: `full_name` (default "Member"), `email`, `query_type` (default "general") and `message`.
3. **Supabase: Save Enquiry**. Creates a row in `training_submissions` with status `new`. The row's `id` is created now and travels to every sub-workflow. It becomes the reference number `PEN-{id}`.
4. **Sub 1: Email Member** (Execute Workflow). Runs the sub-workflow below, waits for it to finish and receives its data back.
5. **Sub 2: Notify Admin** (Execute Workflow). Same pattern.
6. **Sub 3: Update Status** (Execute Workflow). Same pattern.
7. **Respond: Done** (Respond to Webhook). Replies with `ok`, the reference (`PEN-{id}`), the member's name, the email address used and the final status.

All three Execute Workflow nodes use mode Run Once for Each Item and have Wait for Sub-Workflow Completion switched on.

Sub-workflow 1: Email Member

1. **Start: From Main** (Execute Workflow Trigger, accepting all data). Runs only when the main workflow calls it.
2. **Gmail: Acknowledge Member**. Sends the member an acknowledgement quoting `PEN-{id}`, with a promised reply time.
3. **Code: Return Data**. Returns the original enquiry data (`return $('Start - From Main').all()`), because Gmail's own output would otherwise replace it. The main workflow therefore still has the `id` and details for the next sub-workflow.

Sub-workflow 2: Notify Admin

1. **Start: From Main** (Execute Workflow Trigger).
2. **Gmail: Alert Admin**. Sends an internal alert with the reference, member name, email, query type and message to the admin inbox. The admin address is typed into this node and can be changed there.
3. **Code: Return Data**. Passes the enquiry data back, as in Sub 1.

Sub-workflow 3: Update Status in Supabase

1. **Start: From Main** (Execute Workflow Trigger).
2. **Supabase: Update Status**. Operation Update on `training_submissions`, filter `id` equals the incoming `id`, and sets `status` to `completed`. It changes the same row the main workflow created; it does not add a new one.

Import order

Sub-workflows get a new id when imported, so the main workflow cannot find them until you point it at them.

1. Import the three sub-workflows first and save each one.
2. Import the main workflow.
3. Open each Execute Workflow node in the main workflow and choose the matching sub-workflow from the list (Sub 1, Sub 2, Sub 3).
4. Publish all four workflows.

Test (POST to `/training-pension-enquiry`):

```
{
  "full_name": "Sam Taylor",
  "email": "your-email@example.com",
  "query_type": "transfer",
  "message": "I want to transfer my pension"
}
```

Expected:

- Reply: { "ok": true, "reference": "PEN-5", "member": "Sam Taylor", "email_sent_to": "your-email@example.com", "status": "completed" } (the number varies).
- Two emails: the member acknowledgement and the admin alert.
- One row in `training_submissions`, saved as `new` and then updated to `completed`.

Demo tip: keep the row open in Supabase while the workflow runs. It appears as `new` and then switches to `completed`, with the same `id` throughout.

Key lesson: the data a sub-workflow returns is the output of its last node. That is why the two email sub-workflows end with a Code node that returns the original data, and why Sub 3 can find the right row by `id`.

Teaching tips

Teach the workflows in numeric order: each adds one or two ideas on top of the previous one.

Order	Workflow	New idea for students
1	Webhook Intake & Save	Data comes in, gets cleaned, gets stored
2	Router by Category	One input, several paths (Switch)
3	Lookup & Conditional Email	A yes/no decision (IF) and the Always Output Data setting
4	Send Confirmation Email	Sending email, and reaching back to earlier nodes
5	External API Call	Talking to an outside service (HTTP Request) and custom logic (Code)
6	Pension Enquiry	Splitting work into sub-workflows

Run each one in test mode first. Click Listen for test event, send the request from Postman, then click through each node to show its input and output. Students see the data change at every step.

Common errors and fixes

Symptom	Likely cause	Fix
404 ... webhook is not registered	The test URL was used without clicking Listen for test event, or the production URL was used while the workflow is not published	Click Listen first, or publish the workflow
Every field comes out empty or "Unknown"	The body was sent as Text, or in the Params tab	Use Body, raw, JSON in Postman
A node shows a red credential warning after import	The imported credential does not exist on this n8n	Open the node and select the correct Gmail or Supabase account
The workflow stops silently after a Supabase lookup	The search found nothing and Always Output Data is off	Switch Always Output Data on (Workflow 3)
A later node cannot find the email or name	The previous node (Gmail, HTTP, Supabase) replaced \$json	Use \$('Node Name').item.json.field to reach back
The main workflow in Workflow 6 cannot find a sub-workflow	The sub-workflows were imported after the main one, or not selected	Import the subs first, then pick each one in its Execute Workflow node
Supabase calls fail or time out	The free-tier project is paused after inactivity	Restore the project in the Supabase dashboard, wait a couple of minutes
Workflow 5 returns 429	The free Agify limit of about 100 requests per day was reached	Wait until the next day, or use a different name less often

Files in the handover

- 00_supabase_setup.sql : run first.
- 01_Webhook_Intake_Save.json to 05_External_API_Call.json : import in any order.
- 06_Sub_Email_Member.json , 06_Sub_Notify_Admin.json , 06_Sub_Update_Status.json : import before the main workflow.
- 06_Pension_Enquiry_MAIN.json : import last.

Each workflow carries a sticky note describing its flow and a ready-to-use test request, so students can follow along inside n8n itself.